Chapter 5

APIs and Packages

R. Heller · Date: 2026-05-09 · Last changed: 2026-10-31

5.1 Learning objectives

After completing this chapter, you will be able to:

- Query the OpenAlex API using `openalexR` to retrieve works, authors, institutions, and concepts
- Apply filters, pagination, and sampling to control the size and scope of API results
- Deduplicate records by DOI and clean author and affiliation metadata
- Cache API responses locally so repeated book builds do not re-query the API
- Use `rcrossref` and `pubmedR` as complementary data sources

5.2 Setup

```
library(tidyverse)
library(openalexR)
library(glue)

set.seed(20260509)

source(here::here("R", "api_helpers.R"))
```

```
source(here::here("R", "utils.R"))
source(here::here("R", "sci_palette.R"))
```

5.3 Conceptual background

Bibliometric research begins with data. The quality and completeness of a bibliographic corpus determines the validity of every downstream analysis — from citation counts to network maps to topic models.

Historically, Web of Science and Scopus have dominated as data sources for scientometric research. Both require institutional subscriptions and impose restrictions on bulk data use. The launch of **OpenAlex** in 2022 changed this landscape fundamentally (Priem et al., 2022). OpenAlex is a fully open index of over 250 million scholarly works, built from Crossref, PubMed, institutional repositories, and other open sources. It provides a free REST API with no authentication required, making it the ideal backbone for reproducible bibliometric research.

OpenAlex organises its data around five entity types: **works** (publications), **authors**, **sources** (journals, repositories), **institutions**, and **topics** (hierarchical subject tags). Each entity has a unique OpenAlex ID and can be queried via filters on metadata fields such as publication date, citation count, open-access status, and institutional affiliation.

The `openalexR` package provides a tidy R interface to the OpenAlex API. It handles pagination, rate limiting, and result parsing, returning clean tibbles ready for analysis. For metadata not covered by OpenAlex — such as reference lists and funding information — `rcrossref` provides access to Crossref's extensive metadata registry, and `pubmedR` covers the biomedical literature indexed in PubMed/MEDLINE.

5.4 Worked example

We build a small corpus of recent scientometrics research, demonstrating the full acquisition-to-clean-data pipeline.

5.4.1 Querying works from OpenAlex

The provider function is `openalexR::oa_fetch()`. Runnable examples use the book's `fetch_openalex()` wrapper, sourced above, to cache responses by their exact arguments. Set `SCIENTOMETRICS_OFFLINE=1`

to reuse existing OpenAlex caches without requesting new data; a missing cache stops with an explicit error. Other providers have their own network requirements. We search for works published in the journal Scientometrics between 2020 and 2023, sampling 200 records for a manageable demonstration.

```
works <- fetch_openalex(
  entity = "works",
  primary_location.source.id = "S148561398",
  from_publication_date = "2020-01-01",
  to_publication_date = "2023-12-31",
  options = list(sample = 200, seed = 42)
)

glimpse(works)
```

```
#> Rows: 200
#> Columns: 45
#> $ id                            <chr>
 ↪  "https://openalex.org/W3157843452", "htt~
#> $ title                         <chr> "Scientific knowledge
 ↪  production and eco~
#> $ display_name                  <chr> "Scientific knowledge
 ↪  production and eco~
#> $ authorships                   <list> [<tbl_df[2 x 8]>],
 ↪  [<tbl_df[2 x 8]>], [~
#> $ abstract                      <chr> NA, NA, "During the
 ↪  previous Ebola and Z~
#> $ doi                           <chr>
 ↪  "https://doi.org/10.1007/s11192-021-0397~
#> $ publication_date             <date> 2021-04-26, 2021-11-08,
 ↪  2021-04-26, 202~
#> $ publication_year              <int> 2021, 2021, 2021, 2021,
 ↪  2020, 2020, 2020~
#> $ relevance_score               <dbl> 0.999, 0.998, 0.998,
 ↪  0.997, 0.997, 0.997~
#> $ fwci                          <dbl> 1.445, 2.496, 16.402,
 ↪  3.218, 2.548, 12.7~
#> $ cited_by_count                <int> 17, 11, 56, 12, 10, 48,
 ↪  1, 17, 28, 15, 6~
#> $ counts_by_year                <list> [<data.frame[6 x 2]>],
 ↪  [<data.frame[3 x~
#> $ ids                           <list>
 ↪  <"https://openalex.org/W3157843452", "h~
#> $ type                          <chr> "article", "article",
 ↪  "article", "articl~
#> $ is_oa                         <lgl> FALSE, FALSE, TRUE,
 ↪  FALSE, TRUE, FALSE, ~
#> $ is_oa_anywhere                <lgl> TRUE, FALSE, TRUE,
 ↪  FALSE, TRUE, FALSE, F~
```

```
#> $ oa_status                     <chr> "green", "closed",
 ↪  "hybrid", "closed", "~
#> $ oa_url                        <chr>
 ↪  "http://hdl.handle.net/11336/139054", NA~
#> $ any_repository_has_fulltext   <lgl> TRUE, FALSE, TRUE,
 ↪  FALSE, FALSE, FALSE, ~
#> $ source_display_name           <chr> "Scientometrics",
 ↪  "Scientometrics", "Sci~
#> $ source_id                     <chr>
 ↪  "https://openalex.org/S148561398", "http~
#> $ issn_l                        <chr> "0138-9130",
 ↪  "0138-9130", "0138-9130", "~
#> $ host_organization             <chr>
 ↪  "https://openalex.org/P4310320108", "htt~
#> $ host_organization_name        <chr> "Springer Nature
 ↪  (Netherlands)", "Spring~
#> $ landing_page_url              <chr>
 ↪  "https://doi.org/10.1007/s11192-021-0397~
#> $ pdf_url                       <chr> NA, NA,
 ↪  "https://link.springer.com/conte~
#> $ license                       <chr> NA, NA, "other-oa", NA,
 ↪  "cc-by", NA, NA,~
#> $ version                       <chr> "publishedVersion",
 ↪  "publishedVersion", ~
#> $ referenced_works              <list>
 ↪  <"https://openalex.org/W175573758", "ht~
#> $ referenced_works_count        <int> 61, 77, 44, 47, 18, 31,
 ↪  21, 75, 44, 47, ~
#> $ related_works                 <list>
 ↪  <"https://openalex.org/W4415584083", "h~
#> $ concepts                     <list> [<data.frame[25 x 5]>],
 ↪  [<data.frame[21~
#> $ topics                        <list> [<tbl_df[12 x 5]>],
 ↪  [<tbl_df[12 x 5]>],~
#> $ keywords                     <list> [<data.frame[10 x 3]>],
 ↪  [<data.frame[9 ~
#> $ is_paratext                   <lgl> FALSE, FALSE, FALSE,
 ↪  FALSE, FALSE, FALSE~
#> $ is_retracted                  <lgl> FALSE, FALSE, FALSE,
 ↪  FALSE, FALSE, FALSE~
#> $ language                      <chr> "en", "en", "en", "en",
 ↪  "en", "en", "en"~
#> $ sustainable_development_goals <list> NA, NA, NA, NA,
 ↪  [<data.frame[1 x 3]>], ~
#> $ awards                        <list> NA, NA,
 ↪  <"https://openalex.org/G3690828~
#> $ funders                       <list> NA, NA, [<data.frame[2
 ↪  x 3]>], [<data.f~
#> $ apc                           <list> [<data.frame[2 x 5]>],
 ↪  [<data.frame[2 x~
```

```
#> $ first_page                    <chr> "4565", "9477", "4975",
 ↪  "8653", "1783", ~
#> $ last_page                     <chr> "4587", "9498", "4990",
 ↪  "8672", "1800", ~
#> $ volume                        <chr> "126", "126", "126",
 ↪  "126", "125", "126"~
#> $ issue                         <chr> "6", "12", "6", "10",
 ↪  "2", "3", "3", "6"~
```

The result is a tibble where each row is a work and columns include `id`, `display_name`, `publication_date`, `cited_by_count`, `doi`, and nested list-columns for authors and concepts.

5.4.2 Inspecting the results

```
works |>
  select(display_name, publication_date, cited_by_count, doi) |>
  arrange(desc(cited_by_count)) |>
  head(10)
```

```
#> # A tibble: 10 x 4
#>    display_name                              publication_date
 ↪  cited_by_count doi
#>    <chr>                                     <date>
 ↪  <int> <chr>
#>  1 The journal coverage of Web of Science~ 2021-03-26
 ↪  2580 http~
#>  2 A tale of two databases: the use of We~ 2020-02-22
 ↪  1875 http~
#>  3 Mapping the sustainable development go~ 2022-04-04
 ↪  92 http~
#>  4 Analysis of the evolution and collabor~ 2020-10-18
 ↪  79 http~
#>  5 A matter of time: publication dates in~ 2020-10-10
 ↪  77 http~
#>  6 The impact of research output on econo~ 2020-03-28
 ↪  73 http~
#>  7 An in-depth analysis of government fun~ 2020-07-13
 ↪  69 http~
#>  8 Visualizing research trends and resear~ 2020-11-19
 ↪  68 http~
#>  9 Which h-index? An exploration within t~ 2020-04-04
 ↪  66 http~
#> 10 The link between countries' economic a~ 2022-03-11
 ↪  65 http~
```

5.4.3 Querying authors

We can also retrieve author-level data. Let us look up authors associated with scientometrics research by filtering on the relevant topic.

```
authors <- fetch_openalex(
  entity = "authors",
  topics.id = "T10102",
  works_count = ">50",
  options = list(sample = 50, seed = 42)
)

authors |>
  select(display_name, works_count, cited_by_count) |>
  arrange(desc(cited_by_count)) |>
  head(10)
```

```
#> # A tibble: 10 x 3
#>    display_name       works_count cited_by_count
#>    <chr>                    <int>          <int>
#>  1 Hans-Dieter Daniel         220           9117
#>  2 Rüdiger Mutz               121           6318
#>  3 Haak Laurel                296           3356
#>  4 Babak Pourbohloul           53           3169
#>  5 Jordi Suriñach             290           3051
#>  6 Eun-Kyung Chung            186           2888
#>  7 Eti Herman                 103           2647
#>  8 Manuel Goyanes             162           2349
#>  9 Javier Sanz-Valero         277           2219
#> 10 Gali Halevi                 74           1859
```

5.4.4 Deduplication by DOI

Real-world corpora often contain duplicates when merging results from multiple queries or sources. The `dedupe_by_doi()` function from our companion package handles this cleanly.

```
works_with_dupes <- bind_rows(works, works[1:10, ])
cat(glue("Before dedup: {nrow(works_with_dupes)} rows\n\n"))
```

```
#> Before dedup: 210 rows
```

```
works_clean <- dedupe_by_doi(works_with_dupes)
cat(glue("After dedup:  {nrow(works_clean)} rows\n"))
```

```
#> After dedup:  200 rows
```

5.4.5 Caching for reproducible builds

The `fetch_openalex()` wrapper in `R/api_helpers.R` automatically caches API responses to disk. On subsequent calls with the same arguments, it reads from cache instead of hitting the API.

```
works_cached <- fetch_openalex(
  entity = "works",
  primary_location.source.id = "S148561398",
  from_publication_date = "2023-01-01",
  to_publication_date = "2023-06-30",
  options = list(sample = 50, seed = 42),
  cache_dir = "_freeze/openalex_cache",
  cache_days = 30
)

nrow(works_cached)
```

```
#> [1] 50
```

5.4.6 Complementary sources: Crossref

For detailed metadata (reference lists, funders, licenses), Crossref is invaluable.

```
# Requires network access — cached output used in book builds
cr_work <- rcrossref::cr_works(doi =
 ↪  "10.1007/s11192-017-2300-7")
cr_work$data |>
  select(title, container.title, deposited,
   ↪  is.referenced.by.count)
```

5.4.7 Visualization

```
works |>
  mutate(year = year(publication_date)) |>
  count(year) |>
  ggplot(aes(x = year, y = n)) +
  geom_col(fill = palette_sci(1)) +
  labs(x = "Publication year", y = "Number of works") +
  theme_sci()
```

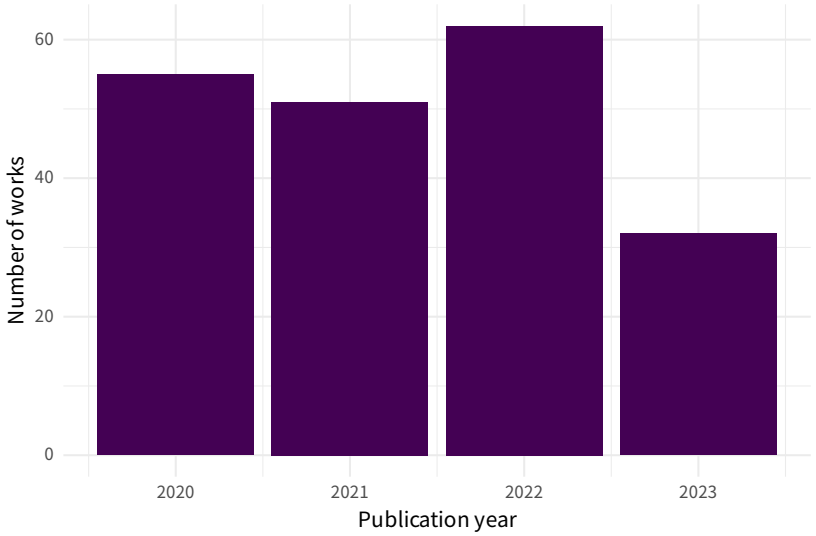


Figure 5.1: Annual publication counts for sampled Scientometrics articles (2020–2023).

5.5 Diagnostics and interpretation

When working with API data, always verify:

- **Completeness**: Compare your result count against the API's reported `meta$count` to confirm you received all expected records. With sampling, the count reflects the sample, not the full set.
- **Coverage dates**: Check `min(works$publication_date)` and `max(works$publication_date)` to confirm your date filters were applied correctly.
- **DOI presence**: Not all works have DOIs. Check `mean(!is.na(works$doi))` to understand the DOI coverage rate in your corpus.
- **Nested columns**: Author and concept columns are list-columns. Use `tidyr::unnest()` carefully and inspect for `NULL` entries before unnesting.

```
cat(glue("Total works: {nrow(works)}\n"))
```

```
#> Total works: 200
```

```
cat(glue("DOI coverage:
 ↪  {scales::percent(mean(!is.na(works$doi)))}\n"))
```

```
#> DOI coverage: 100%
```

```
cat(glue("Date range: {min(works$publication_date)} to
 ↪  {max(works$publication_date)}\n"))
```

```
#> Date range: 2020-01-01 to 2023-12-27
```

5.6 Limitations and responsible use

5.7 Limitations and responsible use

- **Coverage bias**: OpenAlex is strongest in English-language, peer-reviewed journal articles. Conference proceedings, books, and non-English literature may be underrepresented. Always report the data source and its known coverage limitations.
- **Metadata quality**: Author names and affiliations are parsed algorithmically and may contain errors, especially for names with diacritics or non-Latin scripts.
- **Temporal lag**: OpenAlex updates continuously but may lag behind primary databases by days or weeks. Very recent publications may be missing.
- **API rate limits**: OpenAlex allows 10 requests per second for unauthenticated users (100/s with a polite pool email). Respect these limits to avoid being throttled.
- **Reproducibility**: API results can change as OpenAlex updates its data. Always cache results and document the date of data collection for reproducibility (Hicks et al., 2015).

5.8 Common pitfalls

5.9 Common pitfalls

- **Not caching**: Running `oa_fetch()` directly in code chunks without caching means every `quarto render` hits the API, producing different results each time and risking rate limits.

- **Ignoring pagination**: `oa_fetch()` handles pagination automatically, but very large queries (>10,000 works) can time out. Use filters to narrow your query or use cursor-based pagination.
- **Confusing sampling with filtering**: `options = list(sample = 200)` returns a random sample from the full result set, not the first 200 records. This is useful for demonstrations but not for exhaustive analyses.
- **Treating OpenAlex IDs as permanent**: While generally stable, OpenAlex IDs can change when entities are merged. Use DOIs as the primary identifier for works.
- **Merging sources without deduplication**: When combining results from OpenAlex and Crossref, records may overlap. Always deduplicate by DOI before analysis.

5.10 Exercises

1. **Fetch works by institution.** Use `oa_fetch()` to retrieve works affiliated with your own institution. How many works does OpenAlex index for it? What is the DOI coverage rate? (Hint: find your institution's OpenAlex ID via *`oa_fetch(entity = "institutions", search = "...")`*)
2. **Compare Crossref metadata.** Pick a DOI from your fetched works and retrieve its metadata from Crossref using `rcrossref::cr_works()`. Compare the citation count reported by OpenAlex vs. Crossref. Why might they differ?
3. **Build a multi-year corpus.** Fetch all works from a journal of your choice for 2015–2023 (without sampling). Plot the annual publication count and the median citation count per year. What trends do you observe?

5.11 Solutions

Solutions are provided in E.

5.12 Further reading

- Priem et al. (2022) — The OpenAlex paper; describes the data model, coverage, and API design.

- Aria and Cuccurullo (2017) — The `bibliometrix` package; an alternative high-level interface for bibliometric data acquisition and analysis.
- Garfield (1955) — The foundational paper on citation indexing, motivating why these databases exist.

5.13 Session info

```
sessionInfo()
```

```
#> R version 4.6.1 (2026-06-24)
#> Platform: x86_64-pc-linux-gnu
#> Running under: Ubuntu 24.04.5 LTS
#>
#> Matrix products: default
#> BLAS:
 ↪  /usr/lib/x86_64-linux-gnu/openblas-pthread/libblas.so.3
#> LAPACK: /usr/lib/x86_64-linux-gnu/openblas-pthread/libopenbl
 ↪  asp-r0.3.26.so;  LAPACK version 3.12.0
#>
#> locale:
#>  [1] LC_CTYPE=C.UTF-8       LC_NUMERIC=C
 ↪  LC_TIME=C.UTF-8
#>  [4] LC_COLLATE=C.UTF-8     LC_MONETARY=C.UTF-8
 ↪  LC_MESSAGES=C.UTF-8
#>  [7] LC_PAPER=C.UTF-8      LC_NAME=C              LC_ADDRESS=C
#> [10] LC_TELEPHONE=C         LC_MEASUREMENT=C.UTF-8
 ↪  LC_IDENTIFICATION=C
#>
#> time zone: UTC
#> tzcode source: system (glibc)
#>
#> attached base packages:
#> [1] stats     graphics  grDevices utils     datasets  methods
 ↪  base
#>
#> other attached packages:
#>  [1] rcrossref_1.2.1 gt_1.4.0        tidytext_0.4.3
 ↪  glue_1.8.1
#>  [5] openalexR_3.1.2 lubridate_1.9.5 forcats_1.0.1
 ↪  stringr_1.6.0
#>  [9] dplyr_1.2.1     purrr_1.2.2     readr_2.2.0
 ↪  tidyr_1.3.2
#> [13] tibble_3.3.1    ggplot2_4.0.3   tidyverse_2.0.0
#>
#> loaded via a namespace (and not attached):
```

```
#>  [1] gtable_0.3.6       xfun_0.62          htmlwidgets_1.6.4
 ↪  lattice_0.22-9
#>  [5] tzdb_0.5.0         vctrs_0.7.3        tools_4.6.1
 ↪  generics_0.1.4
#>  [9] curl_8.1.0         janeaustenr_1.0.0  pkgconfig_2.0.3
 ↪  tokenizers_0.3.0
#> [13] Matrix_1.7-5       RColorBrewer_1.1-3 S7_0.2.2
 ↪  lifecycle_1.0.5
#> [17] compiler_4.6.1     farver_2.1.2       httpuv_1.6.17
 ↪  htmltools_0.5.9
#> [21] SnowballC_0.7.1    yaml_2.3.12        pillar_1.11.1
 ↪  later_1.4.8
#> [25] DT_0.34.0          viridis_0.6.5      mime_0.13
 ↪  stopwords_2.3
#> [29] tidyselect_1.2.1   digest_0.6.39      stringi_1.8.9
 ↪  bookdown_0.48
#> [33] labeling_0.4.3     rprojroot_2.1.1    fastmap_1.2.0
 ↪  grid_4.6.1
#> [37] here_1.0.2         cli_3.6.6          magrittr_2.0.5
 ↪  triebeard_0.4.1
#> [41] crul_1.6.0         utf8_1.2.6         withr_3.0.3
 ↪  scales_1.4.0
#> [45] promises_1.5.0     timechange_0.4.0   rmarkdown_2.32
 ↪  otel_0.2.0
#> [49] gridExtra_2.3.1    hms_1.1.4          shiny_1.14.0
 ↪  evaluate_1.0.5
#> [53] knitr_1.52         miniUI_0.1.2       viridisLite_0.4.3
 ↪  urltools_1.7.3.1
#> [57] rlang_1.3.0        Rcpp_1.1.2         httpcode_0.3.0
 ↪  xtable_1.8-8
#> [61] xml2_1.6.0         jsonlite_2.0.0     rstudioapi_0.19.0
 ↪  plyr_1.8.9
#> [65] R6_2.6.1           systemfonts_1.3.2  fs_2.1.0
```